

法律文书标注平台

类图（修正稿）

基于 SOLID 检查清单整改结果

适配 ER 图 18 个持久化实体

2025

目 录

一、类图总览（按模块分页）

二、ER 图实体映射验证

2.1 实体对照表

三、各类详细定义

3.0 接口层

3.1 持久化实体层

3.2 服务层

3.3 状态模式类

3.4 模板方法模式类

3.5 观察者模式类

3.6 论证图示模块

四、类之间的关系汇总

五、SOLID 整改对照表

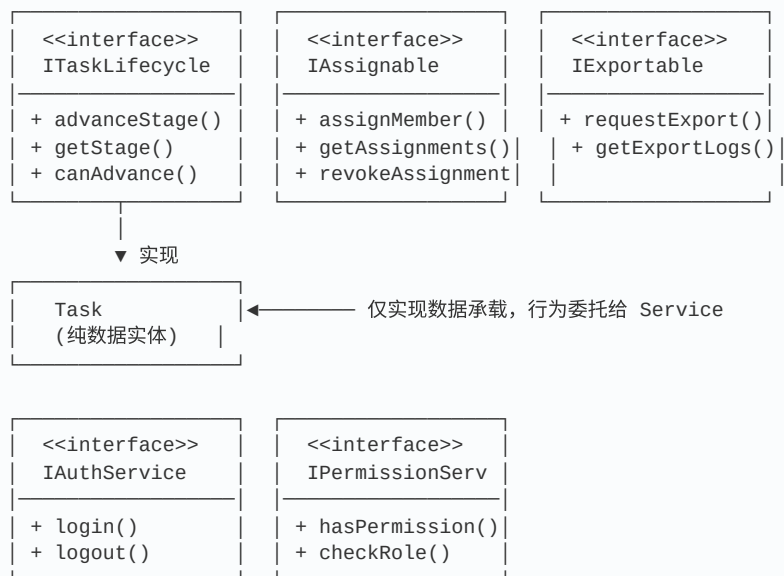
六、设计原则说明（修正后）

一、类图总览

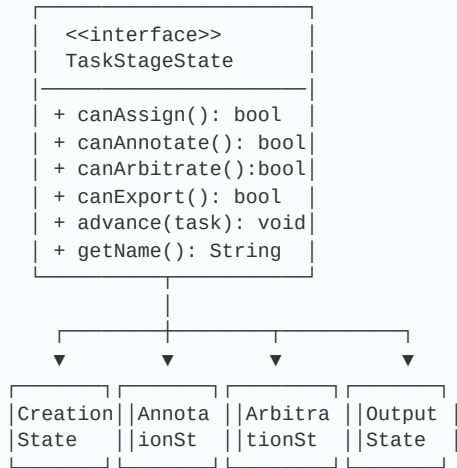
本类图基于 SOLID 检查清单的整改结果，在严格适配 ER 图 18 个持久化实体的前提下，通过增加服务层、接口层和应用设计模式消除全部违规项。

ER 图实体清单（共 18 个，本类图全部覆盖）： 用户表、全局角色表、指南版本表、一级标签表、二级标签表（仅GM）、关系类型表、任务表、全局文件库、任务文档表、任务分配表、命题表、关系表、关系成员明细表、标注结果快照表、裁决结果表、导出日志表、导出文件明细表、任务标签配置历史表。

【接口层 — 接口隔离 + 依赖倒转】

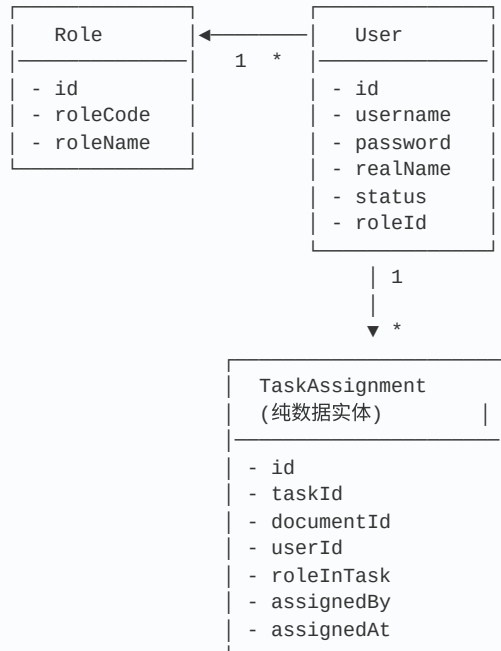


【状态模式 — 开闭原则】

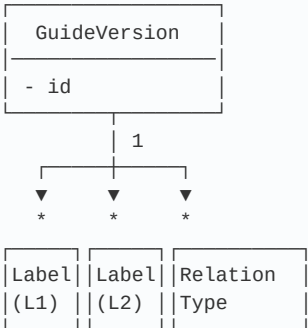


Task 持有 TaskStageState 引用 (组合), 行为委托给当前状态对象

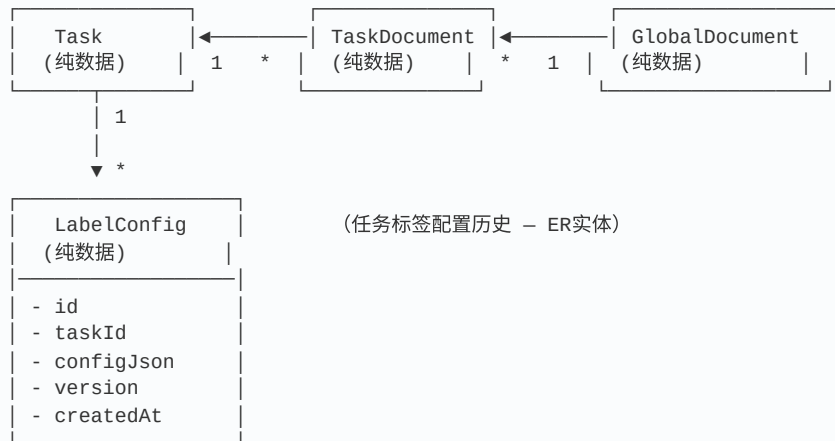
【持久化实体层 — 单一职责（纯数据）— 用户权限模块】



【持久化实体层 — 单一职责（纯数据） — 配置中心模块】



【持久化实体层 — 单一职责（纯数据）— 任务管理模块】



【持久化实体层 — 单一职责（纯数据） — 标注核心模块】

Proposition
(纯数据)

Relation
(纯数据)

RelationMember
(纯数据)

Annotation
(纯数据)

- id
- taskId
- documentId
- userId
- propositionData
- relationData
- submittedAt
- isFinal

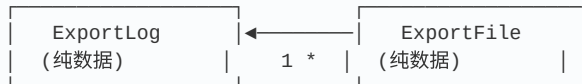
(标注结果快照 — ER实体)
submit/overwrite 已移至 AnnotationService
getPropositions/getRelations 已移至 Converter

【持久化实体层 — 单一职责（纯数据）— 冲突裁决 & 数据导出模块】

【冲突裁决模块】

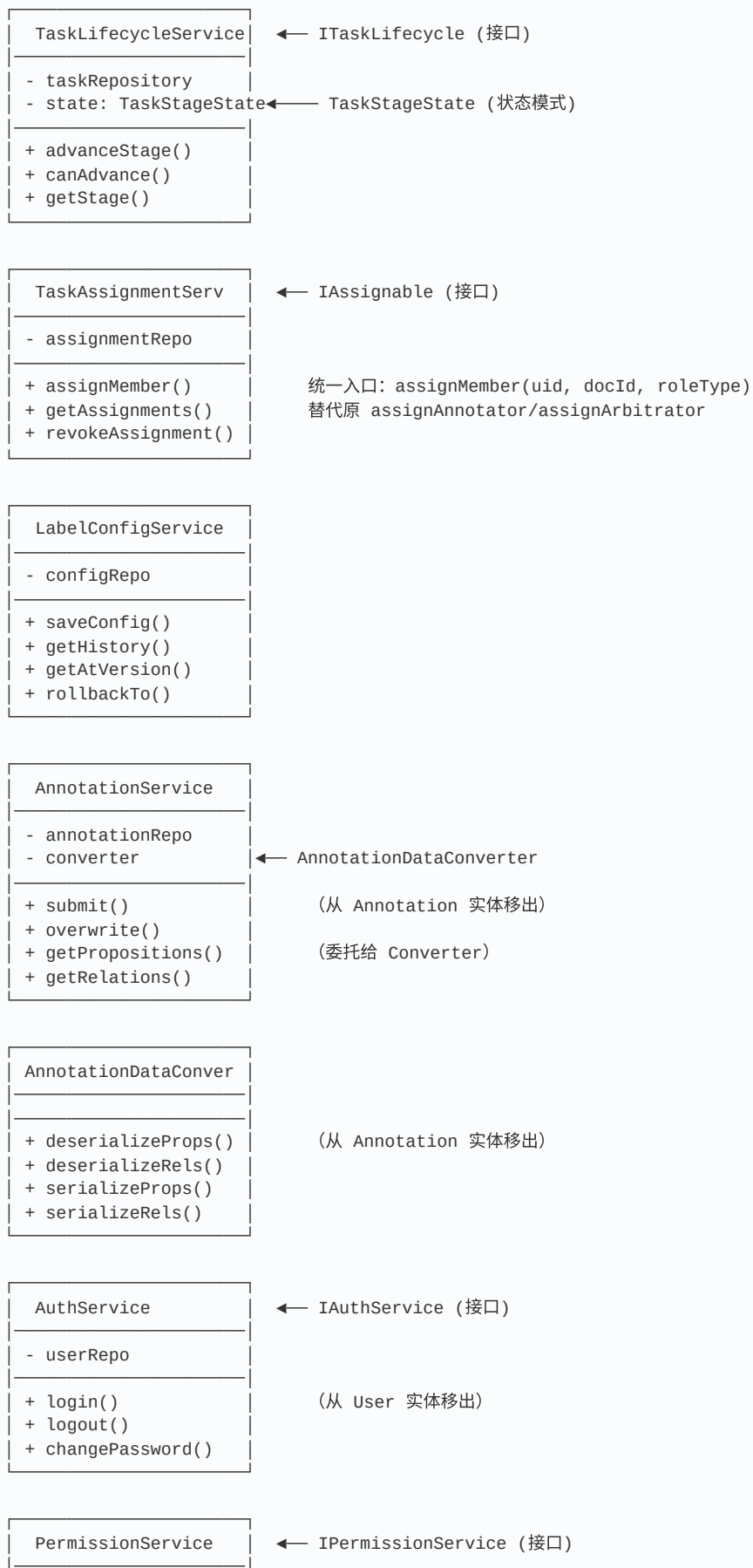


【数据导出模块】

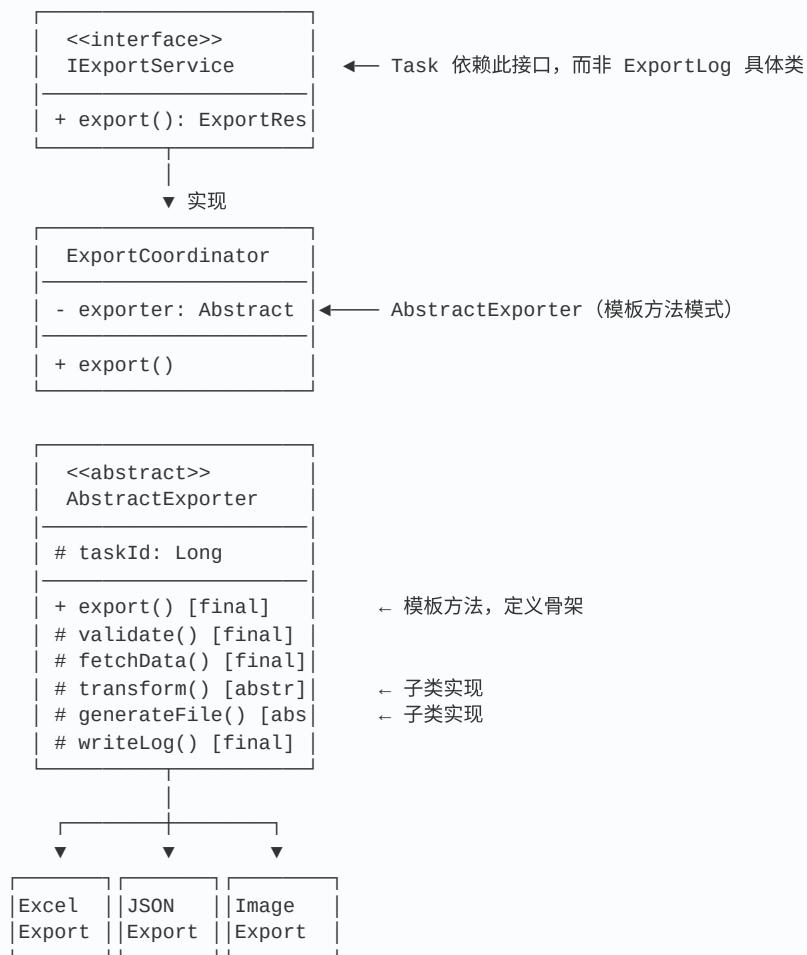


【服务层 — 单一职责 + 依赖倒转】

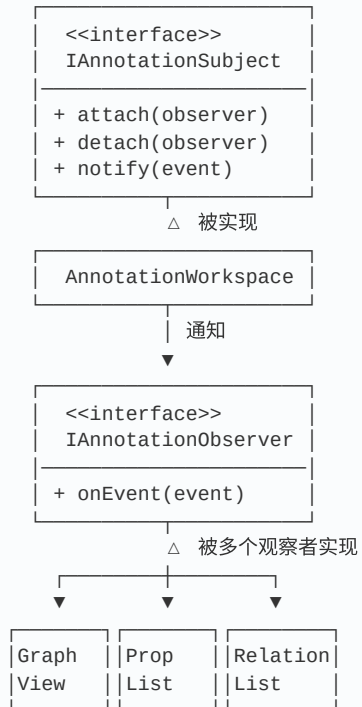
一、类图总览



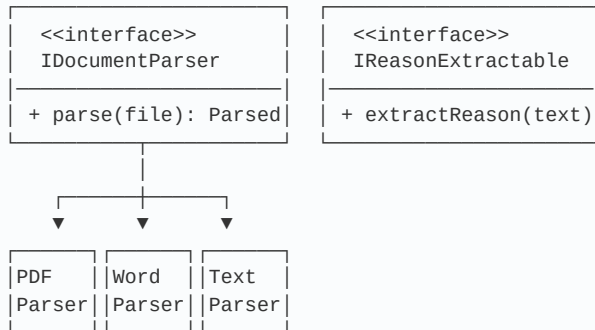
【导出模块 — 模板方法模式 + 依赖倒转】



【观察者模式 — 解耦标注与图示】

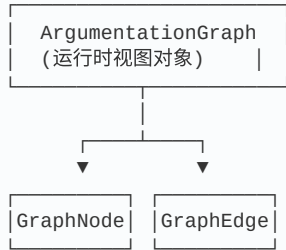


【策略模式 — 文档解析】



通过 `DocumentParserFactory` 运行时选择策略，防止业务代码退化

【论证图示模块 — 运行时对象，无 ER 对应】



二、ER 图实体映射验证

2.1 实体对照表（18 个持久化实体全覆盖）

序号	ER 图实体	类图对应类	状态	职责定位
1	用户表	User	✓	纯数据实体（POJO），仅承载属性
2	全局角色表	Role	✓	纯数据实体（枚举型数据）
3	指南版本表	GuideVersion	✓	纯数据实体
4	一级标签表	Label (level=1)	✓	纯数据实体，通过工厂方法 createL1() 创建
5	二级标签表（仅 GM）	Label (level=2)	✓	纯数据实体，通过工厂方法 createL2() 创建，parentL1Id 必填
6	关系类型表	RelationType	✓	纯数据实体
7	任务表	Task	✓	纯数据实体，行为委托给 Service 和 State
8	全局文件库	GlobalDocument	✓	纯数据实体
9	任务文档表	TaskDocument	✓	纯数据实体
10	任务分配表	TaskAssignment	✓	纯数据实体
11	命题表	Proposition	✓	纯数据实体
12	关系表	Relation	✓	纯数据实体
13	关系成员明细表	RelationMember	✓	纯数据实体
14	标注结果快照表	Annotation	✓	纯数据实体，submit/overwrite 已移至 Service
15	裁决结果表	Arbitration	✓	纯数据实体

二、ER 图实体映射验证

16	导出日志表	ExportLog	✓	纯数据实体
17	导出文件明细表	ExportFile	✓	纯数据实体
18	任务标签配置历史表	LabelConfig	✓	纯数据实体，已从 Task 的 JSON 字段提取为独立类

三、各类详细定义

3.0 接口层（新增 — 接口隔离 + 依赖倒转）

ITaskLifecycle（任务生命周期接口）

方法签名	说明
+ advanceStage(taskId): void	推进任务到下一阶段
+ canAdvance(taskId): boolean	检查是否满足推进条件
+ getCurrentStage(taskId): TaskStage	获取当前阶段

接口隔离：仅包含生命周期相关方法，导出/分配等不相关方法不在此接口。

IAssignable（任务分配接口）

方法签名	说明
+ assignMember(taskId, documentId, userId, roleType): TaskAssignment	统一分配入口，roleType 为枚举
+ getAssignments(taskId): List<TaskAssignment>	获取任务分配列表
+ revokeAssignment(assignmentId): void	撤销分配

开闭原则：新增角色（如"审核员"）只需扩展 TaskRole 枚举，无需新增接口方法。

IExportable（导出接口）

方法签名	说明
+ requestExport(taskId, format, requestedBy): ExportResult	发起导出，返回 ExportResult 抽象结果

+ getExportLogs(taskId): List<ExportLog>

获取导出历史

依赖倒转：Task 依赖 IExportable 接口，而非 ExportLog 具体类。ExportResult 是包含 ExportLog 的封装对象。

IAuthService（认证服务接口）

方法签名	说明
+ login(username, password): AuthToken	用户登录验证
+ logout(token): void	退出登录
+ changePassword(userId, oldPwd, newPwd): boolean	修改密码

单一职责：从 User 实体中剥离，独立为横切关注点服务。

IPermissionService（权限服务接口）

方法签名	说明
+ hasPermission(userId, permissionCode): boolean	权限校验
+ checkRole(userId, roleCode): boolean	角色校验

单一职责：从 User 实体中剥离。

IAnnotationSubject（标注主题接口 — 观察者模式）

方法签名	说明
+ attach(observer: IAnnotationObserver): void	注册观察者
+ detach(observer: IAnnotationObserver): void	移除观察者
+ notify(event: AnnotationEvent): void	通知所有观察者

IAnnotationObserver（标注观察者接口）

方法签名	说明
+ onAnnotationEvent(event: AnnotationEvent): void	接收标注变更事件

IDocumentParser（文档解析接口 — 策略模式）

方法签名	说明
+ parse(file): ParsedDocument	解析文件提取文本

接口隔离：仅保留解析核心方法，extractReasonSection() 分离到 IReasonExtractable。

IReasonExtractable（裁判理由提取接口）

方法签名	说明
+ extractReasonSection(text): String	提取裁判理由/本院认为部分

接口隔离：预防 DocumentParser 变胖。PDF/Word 解析器可同时实现 IDocumentParser 和 IReasonExtractable。

IExportService（导出服务接口 — 依赖倒转）

方法签名	说明
+ export(taskId, format, requestedBy): ExportResult	执行导出

依赖倒转：Task 通过此接口间接使用导出功能，不直接依赖 ExportLog。

3.1 持久化实体层（全部退化为纯数据实体）

User（用户类）— S 原则修正

属性	类型	说明
id	Long	用户ID（主键）
username	String	登录账号（唯一）
password	String	密码（加密存储）
realName	String	真实姓名
phone	String	手机号
status	Integer	状态：1=正常，0=禁用
roleId	Long	全局角色ID（外键→Role.id）
createdAt	DateTime	创建时间
updatedAt	DateTime	更新时间

方法：无（纯数据实体）

修正说明：原 login/logout/changePassword/hasPermission/getTasks 已全部迁移至对应 Service。

Role（角色类）

属性	类型	说明
id	Long	角色ID（主键）
roleCode	String	角色代码：SUPER_ADMIN / TASK_CREATOR / ANNOTATOR / ARBITRATOR
roleName	String	角色显示名称

方法：无

TaskAssignment（任务分配类）— S 原则修正

属性	类型	说明
id	Long	分配记录ID（主键）
taskId	Long	任务ID（外键→Task.id）
documentId	Long	文档ID（外键→TaskDocument.id）
userId	Long	用户ID（外键→User.id）
roleInTask	TaskRole	任务内身份（枚举）
assignedBy	Long	分配者ID（外键→User.id）
assignedAt	DateTime	分配时间

方法：无（纯数据实体）

修正说明：原 `assign/revoke/getUserRoleInTask` 已迁移至 `TaskAssignmentService`。

TaskRole（任务角色枚举）— O 原则修正

ANNOTATOR - 标注员
ARBITRATOR - 裁定者
REVIEWER - 审核员（预留扩展）

开闭原则：新增角色只需增加枚举值，无需修改接口方法。

GuideVersion（指南版本类）

属性	类型	说明
id	Long	版本ID（主键）
versionName	String	版本名称
description	String	版本说明
isActive	Boolean	是否为当前启用版本
createdBy	Long	创建人ID（外键→User.id）
createdAt	DateTime	创建时间

方法：无（纯数据实体，配置型数据）

Label（标签类）— L 原则修正

属性	类型	说明
id	Long	标签ID（主键）
guideVersionId	Long	所属指南版本ID
name	String	标签全称
abbr	String	标签简称
sortOrder	Integer	显示顺序
parentL1Id	Long	所属一级标签ID（二级标签特有）
level	Integer	标签层级：1=一级，2=二级

方法：无（纯数据实体）

工厂方法（保证 L 原则）：

工厂方法	说明
<code>+ createL1(guideVersionId, name, abbr): Label</code>	创建一级标签（parentL1Id 自动为 null）
<code>+ createL2(guideVersionId, parentL1Id, name, abbr): Label</code>	创建二级标签（parentL1Id 必填）

里氏替换修正：通过工厂方法确保 L1/L2 标签在创建时语义正确，调用方无需前置判断 level。

RelationType（关系类型类）

属性	类型	说明
id	Long	关系类型ID（主键）
guideVersionId	Long	所属指南版本ID
name	String	关系名称
abbr	String	关系简称
isBinary	Boolean	是否为二元关系

sortOrder	Integer	显示顺序
-----------	---------	------

方法：无

Task（任务类）— S/O/D 原则修正

属性	类型	说明
id	Long	任务ID（主键）
title	String	任务名称
description	String	任务描述
status	TaskStatus	任务状态枚举
stage	TaskStage	当前阶段枚举
guideVersionId	Long	使用的指南版本ID（外键→GuideVersion.id）
creatorId	Long	创建者ID（外键→User.id）
createdAt	DateTime	创建时间
stageChangedAt	DateTime	阶段最近变更时间

方法（仅保留核心访问器）：

方法签名	说明
+ getId(): Long	获取任务ID
+ getTitle(): String	获取任务名称
+ getStatus(): TaskStatus	获取状态
+ getStage(): TaskStage	获取阶段

修正说明：

- 原 assignAnnotator/assignArbitrator → 迁移到 TaskAssignmentService.assignMember()
- 原 exportData() → 迁移到 ExportCoordinator.export()，Task 通过 IExportable 接口间接调用
- 原 saveLabelConfig/getLabelConfigHistory → 迁移到 LabelConfigService
- 原 advanceStage/canAdvance → 迁移到 TaskLifecycleService，内部使用状态模式

- 原 labelConfigSnapshot JSON 字段 → 已移除，由 LabelConfig 实体替代

TaskStatus（任务状态枚举）

PENDING - 待分配
ANNOTATING - 标注中
ARBITRATING - 裁决中
COMPLETED - 已完成
EXPORTABLE - 可导出

TaskStage（任务阶段枚举）

CREATION - 创建阶段
ANNOTATION - 标注阶段
ARBITRATION - 裁决阶段
OUTPUT - 输出阶段

GlobalDocument（全局文档类）

属性	类型	说明
id	Long	全局文件ID（主键）
fileName	String	文件名
filePath	String	文件存储路径
fileType	String	文件类型：pdf/docx/txt
extractedText	String	提取的纯文本内容
uploadedById	Long	上传者ID（外键→User.id）
uploadedAt	DateTime	上传时间

方法：无（纯数据实体）

TaskDocument（任务文档类）

属性	类型	说明
id	Long	任务文档ID（主键）
taskId	Long	所属任务ID（外键→Task.id）
sourceType	SourceType	数据来源：GLOBAL / UPLOAD

globalDocId	Long	引用全局文件ID（sourceType=GLOBAL时，外键→GlobalDocument.id）
fileName	String	原始文件名
filePath	String	上传文件存储路径
extractedText	String	最终用于标注的纯文本
extractedRange	String	自动提取范围描述
isManuallyCorrected	Boolean	是否手动修正过提取范围
uploadedAt	DateTime	添加时间

方法：无（纯数据实体）

修正说明：原 createFromGlobal/createFromUpload/correctExtraction/getText 已迁移至 TaskDocumentService。

LabelConfig（任务标签配置历史类）— O/D 原则修正

属性	类型	说明
id	Long	配置历史ID（主键）
taskId	Long	所属任务ID（外键→Task.id）
configJson	JSON	完整的标签配置快照
version	Integer	配置版本号
createdAt	DateTime	创建时间

方法：无（纯数据实体）

修正说明：已从 Task.labelConfigSnapshot JSON 字段提取为独立实体，与 ER 图"任务标签配置历史表"精确映射。业务方法迁移至 LabelConfigService。

Proposition（命题类）

属性	类型	说明
id	Long	命题ID（主键）

三、各类详细定义

documentId	Long	所属文档ID（外键→TaskDocument.id）
sequenceNo	Integer	在文档中的序号
startPos	Integer	起始字符位置
endPos	Integer	结束字符位置
selectedText	String	选中的文本片段
labelPath	String	标签路径
createdBy	Long	创建者ID（外键→User.id）
createdAt	DateTime	创建时间
updatedAt	DateTime	更新时间

方法：无（纯数据实体）

修正说明：原 create/delete/updateLabel/renumber/getRelations 已迁移至 PropositionService。

Relation（关系类）

属性	类型	说明
id	Long	关系ID（主键）
documentId	Long	所属文档ID（外键→TaskDocument.id）
relationType	String	关系类型（S/A/J/M/I）
isNested	Boolean	是否为嵌套关系
parentRelationId	Long	父关系ID（外键→Relation.id）
createdBy	Long	创建者ID（外键→User.id）
createdAt	DateTime	创建时间
updatedAt	DateTime	更新时间

方法：无（纯数据实体）

RelationMember（关系成员类）

属性	类型	说明
id	Long	成员明细ID（主键）
relationId	Long	所属关系ID（外键→Relation.id）
memberType	MemberType	成员类型：P=命题，R=关系
memberId	Long	成员ID
memberRole	MemberRole	角色：SOURCE / TARGET / MEMBER
memberOrder	Integer	参数顺序

方法：无（纯数据实体）

Annotation（标注结果快照类）— S/D 原则修正

属性	类型	说明
id	Long	快照ID（主键）
taskId	Long	所属任务ID（外键→Task.id）
documentId	Long	文档ID（外键→TaskDocument.id）
userId	Long	标注员ID（外键→User.id）
propositionData	JSON	命题数据快照（原始JSON字符串）
relationData	JSON	关系数据快照（原始JSON字符串）
submittedAt	DateTime	提交时间
isFinal	Boolean	是否为最终版本

方法：无（纯数据实体）

修正说明：

- 原 `submit/overwrite` → 迁移至 `AnnotationService`
- 原 `getPropositions/getRelations`（含JSON反序列化）→ 迁移至 `AnnotationDataConverter`
- 实体仅持有原始JSON字符串，不依赖任何序列化框架

Arbitration（裁决类）

三、各类详细定义

属性	类型	说明
id	Long	裁决ID（主键）
taskId	Long	所属任务ID（外键→Task.id）
documentId	Long	文档ID（外键→TaskDocument.id, 唯一）
arbitratorId	Long	裁决者ID（外键→User.id）
finalPropositionData	JSON	最终命题数据快照
finalRelationData	JSON	最终关系数据快照
adoptedFrom	String	采纳来源
arbitratedAt	DateTime	裁决时间
isFinal	Boolean	是否为最终版本

方法：无（纯数据实体）

修正说明：原业务方法迁移至 `ArbitrationService`。

ExportLog（导出日志类）

属性	类型	说明
id	Long	导出ID（主键）
taskId	Long	任务ID（外键→Task.id）
exportType	String	导出格式
filePath	String	压缩包文件路径
fileSize	Long	压缩包大小
status	String	状态
progress	Integer	进度（0-100）
labelVersion	String	导出时标签版本快照
requestedBy	Long	请求导出用户ID（外键→User.id）
createdAt	DateTime	请求时间
completedAt	DateTime	完成时间

方法：无（纯数据实体）

ExportFile（导出文件明细类）

属性	类型	说明
id	Long	文件明细ID（主键）
exportLogId	Long	所属导出日志ID（外键→ExportLog.id）
fileUrl	String	文件访问URL
expiryAt	DateTime	临时文件过期时间

方法：无（纯数据实体）

3.2 服务层（新增 — 单一职责 + 依赖倒转）

TaskLifecycleService（任务生命周期服务）— 实现 ITaskLifecycle

依赖	说明
TaskRepository	任务数据访问
TaskStageState	状态模式对象（组合）

方法签名	说明
+ advanceStage(taskId): void	推进阶段，委托给当前 State 对象
+ canAdvance(taskId): boolean	检查推进条件
+ getCurrentStage(taskId): TaskStage	获取当前阶段
+ addDocument(taskId, doc): void	向任务追加文档

单一职责：仅负责任务生命周期管理。

TaskAssignmentService（任务分配服务）— 实现 IAssignable — O 原则修正

依赖	说明
TaskAssignmentRepository	分配记录数据访问

方法签名	说明
+ assignMember(taskId, documentId, userId, roleType): TaskAssignment	统一分配入口
+ getAssignments(taskId): List<TaskAssignment>	获取分配列表
+ revokeAssignment(assignmentId): void	撤销分配
+ getUserRoleInTask(userId, taskId): TaskRole	查询用户角色

开闭原则修正：原 assignAnnotator() 和 assignArbitrator() 两个硬编码方法，合并为 assignMember(uid, docId, roleType) 统一入口。新增角色只需扩展 TaskRole 枚举。

LabelConfigService（标签配置服务）

依赖	说明
LabelConfigRepository	配置历史数据访问

方法签名	说明
+ saveConfig(taskId, configJson): LabelConfig	保存新配置版本
+ getHistoryByTask(taskId): List<LabelConfig>	获取配置变更历史
+ getVersion(taskId, version): LabelConfig	获取指定版本配置
+ getLatestByTask(taskId): LabelConfig	获取最新配置版本
+ rollbackTo(taskId, version): LabelConfig	回滚到指定版本

单一职责：从 Task 类中剥离，独立管理标签配置历史。

AnnotationService（标注服务）— S/D 原则修正

依赖	说明
AnnotationRepository	标注数据访问
AnnotationDataConverter	数据转换器（JSON ↔ 对象）

方法签名	说明
+ submit(taskId, documentId, userId, props, rels): Annotation	提交标注结果
+ overwrite(annotationId, props, rels): void	覆盖提交
+ getPropositions(annotationId): List<Proposition>	获取命题列表（委托 Converter）
+ getRelations(annotationId): List<Relation>	获取关系列表（委托 Converter）
+ getByDocumentAndUser(docId, userId): Annotation	按文档和用户查询

修正说明： submit/overwrite 从 Annotation 实体移出；getPropositions/getRelations 委托给 Converter，Annotation 实体不再依赖 JSON 序列化细节。

AnnotationDataConverter（标注数据转换器）— D 原则修正

方法签名	说明
+ deserializePropositions(json): List<Proposition>	JSON 反序列化为命题列表
+ deserializeRelations(json): List<Relation>	JSON 反序列化为关系列表
+ serializePropositions(props): String	命题列表序列化为 JSON
+ serializeRelations(rels): String	关系列表序列化为 JSON

依赖倒转修正： JSON 反序列化逻辑从 Annotation 实体中彻底移出。Annotation 实体只持有原始 JSON 字符串，不了解序列化格式。切换序列化方案（如 Protobuf）只需修改此转换器。

AuthService（认证服务）— 实现 IAuthService — S 原则修正

依赖	说明
UserRepository	用户数据访问
PasswordEncoder	密码加密器

方法签名	说明
+ login(username, password): AuthToken	用户登录验证
+ logout(token): void	退出登录
+ changePassword(userId, oldPwd, newPwd): boolean	修改密码

单一职责修正： 从 User 实体中剥离认证逻辑。

PermissionService（权限服务）— 实现 IPermissionService — S 原则修正

依赖	说明
UserRepository	用户数据访问
RoleRepository	角色数据访问

方法签名	说明
+ hasPermission(userId, permissionCode): boolean	权限校验
+ checkRole(userId, roleCode): boolean	角色校验

单一职责修正：从 User 实体中剥离权限逻辑。

TaskQueryService（任务查询服务）— S 原则修正

依赖	说明
TaskRepository	任务数据访问

方法签名	说明
+ getTasksByUser(userId): List<Task>	获取用户参与的任务
+ getCreatedTasks(userId): List<Task>	获取用户创建的任务
+ getTaskDetail(taskId): TaskDetailDTO	获取任务详情

单一职责修正：替代原 `User.getTasks()`。

ArbitrationService（裁决服务）

依赖	说明
ArbitrationRepository	裁决数据访问
AnnotationRepository	标注数据访问

方法签名	说明
+ startArbitration(taskId, documentId, arbitratorId): Arbitration	开始裁决

+ adoptAll(arbitrationId, annotationId): void	全部采纳
+ adoptWithEdit(arbitrationId, props, rels): void	部分采纳并修改
+ confirmFinal(arbitrationId): void	确认最终裁定
+ compareAnnotations(ann1, ann2): List<Conflict>	比对差异

ExportCoordinator（导出协调器）— 实现 IExportService — O/D 原则修正

依赖	说明
AbstractExporter	模板方法模式对象（组合）
ExportLogRepository	导出日志数据访问

方法签名	说明
+ export(taskId, format, requestedBy): ExportResult	执行导出
+ getExportLogs(taskId): List<ExportLog>	获取导出历史

开闭原则修正：新增导出格式只需新增 AbstractExporter 子类，零侵入现有代码。

依赖倒转修正：Task 依赖 IExportService 接口，不直接依赖 ExportLog 具体类。ExportResult 是封装了 ExportLog 的结果对象。

DocumentParserFactory（文档解析工厂）— O 原则修正

方法签名	说明
+ getParser(fileType): IDocumentParser	根据文件类型获取解析策略
+ registerParser(fileType, parser): void	注册新解析器

开闭原则修正：防止业务代码退化做 if-else 判断，确保始终通过接口调用。

3.3 状态模式类（新增 — 开闭原则）

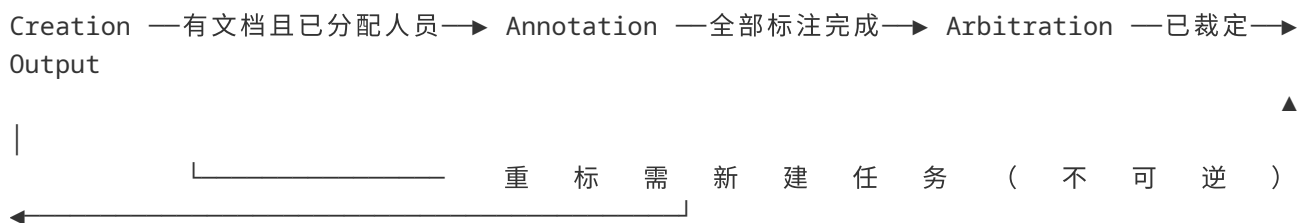
TaskStageState (状态接口)

方法签名	说明
+ canAssign(): boolean	是否允许分配
+ canAnnotate(): boolean	是否允许标注
+ canArbitrate(): boolean	是否允许裁决
+ canExport(): boolean	是否允许导出
+ advance(task): void	推进到下一阶段
+ getName(): String	获取阶段名称

CreationState $\rightarrow$ AnnotationState $\rightarrow$ ArbitrationState $\rightarrow$ OutputState

四个状态类的实现同初稿中的状态模式设计（已验证通过开闭原则），此处省略重复定义。

状态转换规则:



3.4 模板方法模式类（新增 — 开闭原则）

AbstractExporter (抽象导出器)

实现同初稿中的模板方法模式设计（已验证通过开闭原则），此处省略重复定义。

关键设计： `export()` 方法标记为 `final`，子类不可覆盖，确保流程骨架固定。

3.5 观察者模式类（新增 — 解耦）

AnnotationWorkspace（标注工作台 — 实现 IAnnotationSubject）

实现同初稿中的观察者模式设计，此处省略重复定义。

GraphView / PropositionListView（实现 IAnnotationObserver）

运行时视图对象，无 ER 实体对应，同初稿设计。

3.6 论证图示模块（运行时对象，无 ER 对应）

ArgumentationGraph / GraphNode / GraphEdge

同初稿设计，此处省略。

四、类之间的关系汇总

4.1 持久化实体间关系（与 ER 图一致）

关系类型	源类	目标类	基数	对应 ER 图关系
关联	User	Role	N:1	用户表—拥有一全局角色表
关联	User	Task	1:N	用户表—创建—任务表
关联	Task	GuideVersion	N:1	任务表—被引用—指南版本表
关联	Task	TaskDocument	1:N	任务表—包含—任务文档表
关联	TaskDocument	GlobalDocument	N:1	任务文档表—被引用—全局文件库
关联	Task	TaskAssignment	1:N	任务表—分配—任务分配表
关联	TaskAssignment	User	N:1	任务分配表—被分配—用户表
关联	GuideVersion	Label	1:N	指南版本表—定义—一级/二级标签表
关联	Label(L1)	Label(L2)	1:N	一级标签表—包含—二级标签表
关联	GuideVersion	RelationType	1:N	指南版本表—定义—关系类型表
关联	TaskDocument	Proposition	1:N	任务文档表—包含—命题表
关联	TaskDocument	Relation	1:N	任务文档表—包含—关系表
关联	Relation	RelationMember	1:N	关系表—包含成员—关系成员明细表
关联	Proposition	RelationMember	1:N（间接）	命题表—作为成员—关系成员明细表
关联	Relation	Relation	1:N	关系表—自嵌套—关系表
关联	TaskDocument	Annotation	1:N	任务文档表—产生—标注结果快照表
关联	User	Annotation	1:N	用户表—提交—标注结果快照表
关联	TaskDocument	Arbitration	1:1	任务文档表—拥有一裁决结果表
关联	Task	ExportLog	1:N	任务表—发起导出—导出日志表
关联	ExportLog	ExportFile	1:N	导出日志表—包含—导出文件明细表
关联	Task	LabelConfig	1:N	任务表—拥有一任务标签配置历史表

4.2 服务层与实体层关系

关系类型	源类	目标类	说明
依赖	TaskLifecycleService	Task	服务依赖实体数据
依赖	TaskLifecycleService	TaskStageState	组合，状态模式
依赖	TaskAssignmentService	TaskAssignment	服务操作实体
依赖	LabelConfigService	LabelConfig	服务操作实体
依赖	AnnotationService	Annotation	服务操作实体
依赖	AnnotationService	AnnotationDataConverter	组合，数据转换
依赖	ExportCoordinator	AbstractExporter	组合，模板方法模式
依赖	ExportCoordinator	ExportLog	记录导出结果
依赖	AuthService	User	认证服务依赖用户实体
依赖	PermissionService	User / Role	权限服务依赖用户/角色
依赖	TaskQueryService	Task	查询服务依赖任务实体

4.3 接口实现关系

关系类型	实现类	接口	说明
实现	TaskLifecycleService	ITaskLifecycle	生命周期管理
实现	TaskAssignmentService	IAssignable	任务分配
实现	ExportCoordinator	IExportService	导出服务
实现	AuthService	IAuthService	认证
实现	PermissionService	IPermissionService	权限
实现	AnnotationWorkspace	IAnnotationSubject	观察者模式—主题
实现	GraphView / PropListView	IAnnotationObserver	观察者模式—观察者
实现	PdfParser	IDocumentParser	策略模式
实现	WordParser	IDocumentParser	策略模式
实现	TextParser	IDocumentParser	策略模式

四、类之间的关系汇总

实现	PdfParser	IReasonExtractable	接口隔离
实现	WordParser	IReasonExtractable	接口隔离
实现	CreationState / AnnotationState / ArbitrationState / OutputState	TaskStageState	状态模式
实现	ExcelExporter / JsonExporter / ImageExporter	AbstractExporter	模板方法模式

五、SOLID 整改对照表

原则	原违规项	修正措施	修正位置
S-1	Task 5大职责聚合	拆出 <code>TaskLifecycleService</code> 、 <code>TaskAssignmentService</code> 、 <code>LabelConfigService</code> 、 <code>ExportCoordinator</code>	3.2 服务层
S-2	Annotation 混入业务+序列化	拆出 <code>AnnotationService</code> 、 <code>AnnotationDataConverter</code>	3.2 服务层
S-3	User 混入认证+权限+查询	拆出 <code>AuthService</code> 、 <code>PermissionService</code> 、 <code>TaskQueryService</code>	3.2 服务层
O-1	新增导出格式改现有代码	应用模板方法模式： <code>AbstractExporter</code> + 子类	3.4 模板方法模式
O-2	新增角色需加硬编码方法	<code>assignAnnotator/assignArbitrator</code> → <code>assignMember(uid, docId, roleType)</code>	<code>IAssignable</code> 接口
O-3	<code>labelConfigSnapshot</code> JSON死字段	提取 <code>LabelConfig</code> 独立实体	3.1 <code>LabelConfig</code>
O-4	Parser退化风险	增加 <code>DocumentParserFactory</code> 工厂	3.2 <code>DocumentParserFactory</code>
L-1	Label类语义双重性	增加工厂方法 <code>createL1 / createL2</code> 保证创建语义	Label 工厂方法
I-1	Task方法横跨4领域	拆分为 <code>ITaskLifecycle</code> / <code>IAssignable</code> / <code>IExportable</code> 等	3.0 接口层
I-2	Parser未来变胖风险	拆分为 <code>IDocumentParser</code> / <code>IReasonExtractable</code>	3.0 接口层
D-1	Task直接依赖 <code>ExportLog</code>	Task → <code>IExportService</code> → <code>ExportCoordinator</code> → <code>ExportLog</code>	3.0/3.2 接口+服务层

五、SOLID 整改对照表

D-2	Annotation依赖JSON序列化	JSON转换移至 AnnotationDataConverter，实体只存 原始字符串	3.2 AnnotationDataConverter
D-3	Task依赖JSON存储格式	labelConfigSnapshot 移除，改为 LabelConfig 实体	3.1 LabelConfig

六、设计原则说明（修正后）

1. **单一职责（S）**：每个持久化实体类仅承载数据，所有业务行为委托给专门的服务类。
Service 层按功能域拆分，一个 Service 只负责一个功能领域。
2. **开闭原则（O）**：
 - 任务阶段变更通过状态模式实现，新增阶段只需新增 State 子类
 - 导出格式变更通过模板方法模式实现，新增格式只需新增 Exporter 子类
 - 任务角色变更通过枚举扩展，无需修改分配接口
 - 文档解析格式变更通过策略模式 + 工厂实现，新增格式只需注册新 Parser
3. **里氏替换（L）**：Label 工厂方法保证 L1/L2 标签创建时语义正确，调用方可安全使用。
PdfParser/WordParser/TextParser 均可无差别替换 IDocumentParser。
4. **接口隔离（I）**：ITaskLifecycle/IAssignable/IExportable 等接口各司其职，依赖方只依赖所需接口。IDocumentParser 与 IReasonExtractable 分离，防止接口变胖。
5. **依赖倒转（D）**：
 - Task 实体依赖 ITaskLifecycle/IAssignable/IExportable 等抽象接口，不直接依赖具体 Service
 - Service 层依赖 Repository 接口（隐含），不直接依赖具体 DAO 实现
 - Annotation 实体不依赖任何序列化框架，JSON 转换由 AnnotationDataConverter 负责